

# Programming Methodology Lab2

## Modern C++ & STL 2

Lab2

Fall 2025

Hyunwoong Bae (배현웅)

hwbae0326@snu.ac.kr

# STL Documentation & Tutorial

STL documentation and tutorials

- <https://en.cppreference.com/w/cpp/container.html>
- <https://www.geeksforgeeks.org/the-c-standard-template-library-stl/>

Doctors: Googling stuff online does not make you a doctor.

Programmers:



# Outline

- STL in modern C++
  - Vector
    - Container
    - Algorithm
  - Set
  - Map

```
# include <iostream>
# include <string>
# include <vector>
# include <set>
# include <map>
# include <stack>
# include <queue>
```

# Remind

- **Containers**

- Create and store data and objects

- Sequence containers: `array`, `vector`, `list`, `deque`, `forward_list`
    - Container adaptors: `stack`, `queue`, `priority_queue`
    - Associative container: `set`, `multiset`, `map`, `multimap`

## Standard Template Library in C++

### Container

- Sequence Containers
- Associative Containers
- Container Adapters
- Unordered Associative Containers

### Iterator

- `begin()`
- `next()`
- `prev()`
- `advance()`
- `end()`

### Algorithm

- Sorting Algorithms
- Search algorithms
- Non modifying algorithms
- Modifying algorithms
- Numeric algorithms
- Minimum and Maximum operations

# I. Vector

## I.1. Introduction

- **Containers**

- Create and store data and objects

- Sequence containers: array, **vector**, list, deque, forward\_list
- Container adaptors: stack, queue, priority\_queue
- Associative container: set, multiset, map, multimap

### Standard Template Library in C++

#### Container

- Sequence Containers
- Associative Containers
- Container Adapters
- Unordered Associative Containers

#### Iterator

- begin()
- next()
- prev()
- advance()
- end()

#### Algorithm

- Sorting Algorithms
- Search algorithms
- Non modifying algorithms
- Modifying algorithms
- Numeric algorithms
- Minimum and Maximum operations

# I. Vector

## I.1. Introduction

### Sequence Containers:

Array:



->

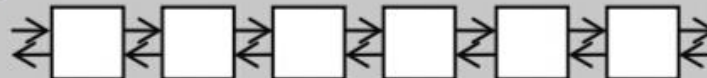
Vector:



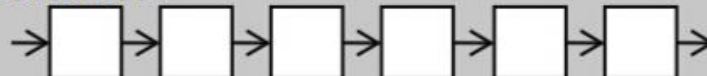
Deque:



List:



Forward-List:



# I.Vector

## I.I.Introduction

- What is the Vector?

- The storage of the vector is handled automatically being expanded and contracted as needed.
- Possible to Insert or erase data

- Syntax

- `vector<data_type> vector_name;`

- Member function categories

- Element access
- Iterators
- Capacity
- Operations



# I. Vector

## I.2. Member Functions

### Member functions

|                              |                                                                                                            |
|------------------------------|------------------------------------------------------------------------------------------------------------|
| (constructor)                | constructs the vector<br>(public member function)                                                          |
| (destructor)                 | destructs the vector<br>(public member function)                                                           |
| <b>operator=</b>             | assigns values to the container<br>(public member function)                                                |
| <b>assign</b>                | assigns values to the container<br>(public member function)                                                |
| <b>get_allocator</b>         | returns the associated allocator<br>(public member function)                                               |
| <b>Capacity</b>              |                                                                                                            |
| <b>empty</b>                 | checks whether the container is empty<br>(public member function)                                          |
| <b>size</b>                  | returns the number of elements<br>(public member function)                                                 |
| <b>max_size</b>              | returns the maximum possible number of elements<br>(public member function)                                |
| <b>reserve</b>               | reserves storage<br>(public member function)                                                               |
| <b>capacity</b>              | returns the number of elements that can be held in currently allocated storage<br>(public member function) |
| <b>shrink_to_fit</b> (C++11) | reduces memory usage by freeing unused memory<br>(public member function)                                  |
| <b>Modifiers</b>             |                                                                                                            |
| <b>clear</b>                 | clears the contents<br>(public member function)                                                            |
| <b>insert</b>                | inserts elements<br>(public member function)                                                               |
| <b>emplace</b> (C++11)       | constructs element in-place<br>(public member function)                                                    |
| <b>erase</b>                 | erases elements<br>(public member function)                                                                |
| <b>push_back</b>             | adds an element to the end<br>(public member function)                                                     |
| <b>emplace_back</b> (C++11)  | constructs an element in-place at the end<br>(public member function)                                      |
| <b>pop_back</b>              | removes the last element<br>(public member function)                                                       |
| <b>resize</b>                | changes the number of elements stored<br>(public member function)                                          |
| <b>swap</b>                  | swaps the contents<br>(public member function)                                                             |





# I.Vector

## I.3. Initialization & Iterator

- `push_back(value)`: It push the elements into a vector from the back
- `begin()`
- `end()`

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  void print_vector(vector<int> &vec)
5  {
6      for (auto i : vec)
7          cout << i << " ";
8
9      cout << endl;
10 }
11 int main()
12 {
13     int sum = 0;
14     // declaration and assignment
15     vector<int> vec;
16     for (int i = 1; i <= 4; i++)
17     {
18         vec.push_back(i);
19     }
20     // print the elements in array
21     cout << "Initial vector: ";
22     print_vector(vec);
23
24     cout << "begin:\t" << &*vec.begin() << endl
25         << "end:\t" << &*vec.end() << endl;
26
27     vec = {1, 2, 3};
28     cout << "Alternative initialization: ";
29     print_vector(vec);
30
31     cout << "begin:\t" << &*vec.begin() << endl
32         << "end:\t" << &*vec.end() << endl;
33
34     return 0;
35 }
```

# I.Vector

## I.3. Initialization & Iterator

- `push_back(value)`: It push the elements into a vector from the back
- `begin()`
- `end()`

```
Initial vector: 1 2 3 4
begin: 0x125e06ce0
end: 0x125e06cf0
Alternative initialization: 1 2 3
begin: 0x125e06ce0
end: 0x125e06cec
```

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  void print_vector(vector<int> &vec)
5  {
6      for (auto i : vec)
7          cout << i << " ";
8
9      cout << endl;
10 }
11 int main()
12 {
13     int sum = 0;
14     // declaration and assignment
15     vector<int> vec;
16     for (int i = 1; i <= 4; i++)
17     {
18         vec.push_back(i);
19     }
20     // print the elements in array
21     cout << "Initial vector: ";
22     print_vector(vec);
23
24     cout << "begin:\t" << &*vec.begin() << endl
25         << "end:\t" << &*vec.end() << endl;
26
27     vec = {1, 2, 3};
28     cout << "Alternative initialization: ";
29     print_vector(vec);
30
31     cout << "begin:\t" << &*vec.begin() << endl
32         << "end:\t" << &*vec.end() << endl;
33
34     return 0;
35 }
```

# I.Vector

## I.3. Exercise #1 – Initialization & Iterator

- What is the Vector?

- The storage of the **vector** is handled automatically being **expanded and contracted** as needed.



```
Max_Size : 4611686018427387903
```

```
Initial array: 1 2
```

```
Size : 2
```

```
Capacity : 2
```

```
After push: 1 2 1 2
```

```
Size : 4
```

```
Capacity : 4
```

```
After push: 1 2 1 2 1 2
```

```
Size : 6
```

```
Capacity : 8
```

```
After resize: 1 2 1
```

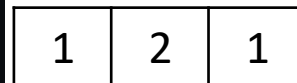
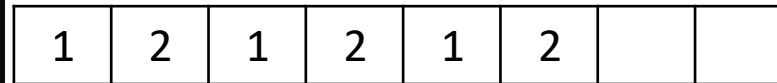
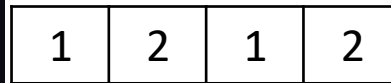
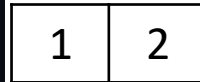
```
Size : 3
```

```
Capacity : 8
```

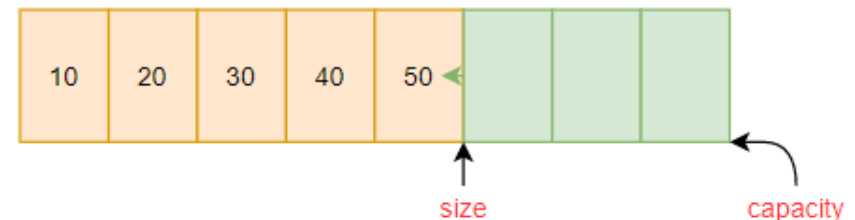
```
After shrink: 1 2 1
```

```
Size : 3
```

```
Capacity : 3
```



Default: expand size by  
current size x2



# I.Vector

## I.3. Exercise #1 – Initialization & Iterator

- Example in [cppreference.com](https://en.cppreference.com)

```
#include <vector>
#include <iostream>

void print(auto rem, const std::vector<int>& c)
{
    for (std::cout << rem; const int el : c)
        std::cout << el << ' ';
    std::cout << '\n';
}

int main()
{
    std::vector<int> c = {1, 2, 3};
    print("The vector holds: ", c);

    c.resize(5);
    print("After resize up to 5: ", c);

    c.resize(2);
    print("After resize down to 2: ", c);

    c.resize(6, 4);
    print("After resize up to 6 (initializer = 4): ", c);
}
```

Output:

```
The vector holds: 1 2 3
After resize up to 5: 1 2 3 0 0
After resize down to 2: 1 2
After resize up to 6 (initializer = 4): 1 2 4 4 4 4
```

# I.Vector

## I.3. Exercise #I – Initialization & Iterator

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void print_vector(vector<int> &vec)
6  {
7      for (auto i : vec)
8          cout << i << " ";
9      cout << endl;
10 }
11
12 void print_vector_info(vector<int> &vec)
13 {
14     cout << "Size : " << vec.size() << endl;
15     cout << "Capacity : " << vec.capacity() << endl;
16 }
17
18 int main()
19 {
20     // Declaration and assign
21     ? vec;
22     for (int i = 1; i <= 2; i++)
23     {
24         ?
25     }
26     cout << "Max_Size : " << vec.max_size() << endl;
27
28     // Print the elements in vector
29     cout << "Initial vector : ";
30     print_vector(vec);
31     print_vector_info(vec);
32
33     // Push elements
34     for (int i = 1; i <= 2; i++)
35     {
36         ?
37     }
38     cout << "After push-1: ";
39     print_vector(vec);
40     print_vector_info(vec);
41
42     // Push elements
43     for (int i = 1; i <= 2; i++)
44     {
45         ?
46     }
47     cout << "After push-2: ";
48     print_vector(vec);
49     print_vector_info(vec);
50
51     // Resize the vector size to 3
52     ?
53     cout << "After resize: ";
54     print_vector(vec);
55     print_vector_info(vec);
56
57     // Shrinks the vector
58     ?
59     cout << "After shrink_to_fit: ";
60     print_vector(vec);
61     print_vector_info(vec);
62
63     return 0;
64 }
```

```
Max_Size : 4611686018427387903
Initial vector : 1 2
Size : 2
Capacity : 2
After push-1: 1 2 1 2
Size : 4
Capacity : 4
After push-2: 1 2 1 2 1 2
Size : 6
Capacity : 8
After resize: 1 2 1
Size : 3
Capacity : 8
After shrink_to_fit: 1 2 1
Size : 3
Capacity : 3
```

# I.Vector

## I.3. Exercise #I – Initialization & Iterator

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void print_vector(vector<int> &vec)
6  {
7      for (auto i : vec)
8          cout << i << " ";
9      cout << endl;
10 }
11
12 void print_vector_info(vector<int> &vec)
13 {
14     cout << "Size : " << vec.size() << endl;
15     cout << "Capacity : " << vec.capacity() << endl;
16 }
17
18 int main()
19 {
20     // Declaration and assign
21     vector<int> vec;
22
23     for (int i = 1; i <= 2; i++)
24     {
25         vec.push_back(i);
26     }
27     cout << "Max_Size : " << vec.max_size() << endl;
28
29     // Print the elements in vector
30     cout << "Initial vector : ";
31     print_vector(vec);
32     print_vector_info(vec);
33
34     // Push elements
35     for (int i = 1; i <= 2; i++)
36     {
37         vec.push_back(i);
38     }
39     cout << "After push-1: ";
40     print_vector(vec);
41     print_vector_info(vec);
42
43     // Push elements
44     for (int i = 1; i <= 2; i++)
45     {
46         vec.push_back(i);
47     }
48     cout << "After push-2: ";
49     print_vector(vec);
50     print_vector_info(vec);
51
52     // Resize the vector size to 3
53     vec.resize(3);
54     cout << "After resize: ";
55     print_vector(vec);
56     print_vector_info(vec);
57
58     // Shrinks the vector
59     vec.shrink_to_fit();
60     cout << "After shrink_to_fit: ";
61     print_vector(vec);
62     print_vector_info(vec);
63
64     return 0;
65 }
```

```
Max_Size : 4611686018427387903
Initial vector : 1 2
Size : 2
Capacity : 2
After push-1: 1 2 1 2
Size : 4
Capacity : 4
After push-2: 1 2 1 2 1 2
Size : 6
Capacity : 8
After resize: 1 2 1
Size : 3
Capacity : 8
After shrink_to_fit: 1 2 1
Size : 3
Capacity : 3
```

# I.Vector

## I.3. Exercise #2 – Modifiers

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void print_vector(vector<int> &vec)
6  {
7      for (auto i : vec)
8          cout << i << " ";
9      cout << endl;
10 }
11
12 int main()
13 {
14     int sum = 0;
15     // declaration and assignment
16     vector<int> vec;
17     for (int i = 1; i <= 4; i++)
18     {
19         vec.push_back(i);
20     }
21
22     // print the elements in vector
23     cout << "Initial vector: ";
24     print_vector(vec);
25
26     ?
27     {
28         ?
29         ?
30     }
31     cout << "Sum of elements: " << sum << endl;
```

Sum every element and pop all

```
34     ?
35
36     cout << "after assign: ";
37     print_vector(vec);
38
39     // inserts 5 at the second position
40     ?
41     cout << "after insert: ";
42     print_vector(vec);
43
44     // removes the first element
45     ?
46     cout << "after erase: ";
47     print_vector(vec);
48
49     // emplace 20 at back
50     ?
51     cout << "after emplace_back: ";
52     print_vector(vec);
53
54     // erase the vector
55     ?
56     cout << "Vector size after erase: " << vec.size() << endl;
57
58     return 0;
59 }
```

```
Initial vector: 1 2 3 4
Sum of elements: 10
after assign: 10 10 10
after insert: 10 5 10 10
after erase: 5 10 10
after emplace_back: 5 10 10 20
Vector size after erase: 0
```

# I. Vector

## I.3. Exercise #2 – Modifiers

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void print_vector(vector<int> &vec)
6  {
7      for (auto i : vec)
8          cout << i << " ";
9      cout << endl;
10 }
11
12 int main()
13 {
14     int sum = 0;
15     // declaration and assignment
16     vector<int> vec;
17     for (int i = 1; i <= 4; i++)
18     {
19         vec.push_back(i);
20     }
21
22     // print the elements in vector
23     cout << "Initial vector: ";
24     print_vector(vec);
25
26     while (!vec.empty())
27     {
28         sum += vec.back();
29         vec.pop_back();
30     }
31     cout << "Sum of elements: " << sum << endl;
```

Sum every element  
and pop all

```
34     vec.assign(3, 10);
35
36     cout << "after assign: ";
37     print_vector(vec);
38
39     // inserts 5 at the second position
40     vec.insert(vec.begin() + 1, 5);
41     cout << "after insert: ";
42     print_vector(vec);
43
44     // removes the first element
45     vec.erase(vec.begin());
46     cout << "after erase: ";
47     print_vector(vec);
48
49     // emplace 20 at back
50     vec.emplace_back(20);
51     cout << "after emplace_back: ";
52     print_vector(vec);
53
54     // erase the vector
55     vec.clear();
56     cout << "Vector size after erase: " << vec.size() << endl;
57
58     return 0;
59 }
```

```
Initial vector: 1 2 3 4
Sum of elements: 10
after assign: 10 10 10
after insert: 10 5 10 10
after erase: 5 10 10
after emplace_back: 5 10 10 20
Vector size after erase: 0
```



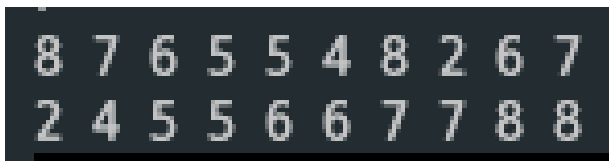
# Outline

- STL in modern C++
  - Vector
    - Container
    - Algorithm
  - Set
  - Map

# I. Vector

## I.4. Sorting

- Remind:
  - We have learned the `sort()` function in the previous chapter



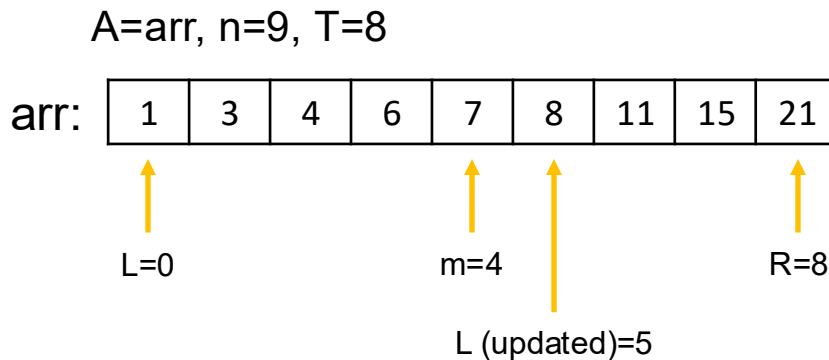
8 7 6 5 5 4 8 2 6 7  
2 4 5 5 6 6 7 7 8 8

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  using namespace std;
5
6  void print_vector(vector<int> &vec)
7  {
8      for (auto i : vec)
9          cout << i << " ";
10     cout << endl;
11 }
12
13 int main()
14 {
15     srand((int)time(NULL));
16     // declaration and assignment
17     vector<int> vec;
18     for (int i = 1; i <= 10; i++)
19     {
20         vec.push_back(rand() % 10);
21     }
22
23     print_vector(vec);
24     sort(vec.begin(), vec.end());
25     print_vector(vec);
26
27     return 0;
28 }
```

# I.Vector

## I.5. Searching – Binary search

- What is the binary search?
  - A search algorithm that finds the position of a target value within a sorted array
  - $O(\log n)$  for average and worst case
- Algorithm



```
function binary_search(A, n, T) is
    L := 0
    R := n - 1
    while L ≤ R do
        m := floor((L + R) / 2)
        if A[m] < T then
            L := m + 1
        else if A[m] > T then
            R := m - 1
        else:
            return m
    return unsuccessful
```

# I.Vector

## I.5. Searching – Exercise #3

- Note
  - Use iterator
    - Do not use get, at, and [] operator.
  - Return end() for the unsuccessful case

```
function binary_search(A, n, T) is
  L := 0          <- Init by iterator
  R := n - 1      <- Init by iterator
  while L ≤ R do
    m: iterator m := floor((L + R) / 2)
    if A[m] < T then
      L := m + 1
    else if A[m] > T then
      R := m - 1
    else:
      return m
  return unsuccessful  Return end()
```

```
1 3 2 8 9 10 17 15 31
1 2 3 8 9 10 15 17 31
index : 5, value : 10
```

```
1 #include <iostream>
2 #include <algorithm>
3 #include <vector>
4 using namespace std;
5
6 void print_vector(vector<int> &vec)
7 {
8     for (auto i : vec)
9         cout << i << " ";
10    cout << endl;
11 }
12
13 vector<int>::iterator my_binary_search(vector<int> &A, int T)
14 {
15     [Redacted]
16 }
17
18 int main()
19 {
20     // declaration and assignment
21     vector<int> vec = {1, 3, 2, 8, 9, 10, 17, 15, 31};
22
23     print_vector(vec);
24     sort(vec.begin(), vec.end());
25     print_vector(vec);
26
27     vector<int>::iterator it = my_binary_search(vec, 10);
28     cout << "index : " << it - vec.begin() << ", value : " << *it << endl;
29     return 0;
30 }
```

# I.Vector

## I.5. Searching – Exercise #3

- Note
  - Use iterator
    - Do not use get, at, and [] operator.
  - Return end() for the unsuccessful case

```
function binary_search(A, n, T) is
  L := 0          <- Init by iterator
  R := n - 1      <- Init by iterator
  while L ≤ R do
    m := floor((L + R) / 2)
    if A[m] < T then
      L := m + 1
    else if A[m] > T then
      R := m - 1
    else:
      return m
  return unsuccessful
```

Return end()

```
1 3 2 8 9 10 17 15 31
1 2 3 8 9 10 15 17 31
index : 5, value : 10
```

```
1 #include <iostream>
2 #include <algorithm>
3 #include <vector>
4 using namespace std;
5
6 void print_vector(vector<int> &vec)
7 {
8     for (auto i : vec)
9         cout << i << " ";
10    cout << endl;
11 }
12
13 vector<int>::iterator my_binary_search(vector<int> &A, int T)
14 {
15     // Write your code here
16     // Do not use get, at and [] operator
17     auto l = A.begin();
18     auto r = A.end() - 1;
19
20     while (l <= r)
21     {
22         auto m = l + int((r - l + 1) / 2);
23         if (*m < T)
24             l = m + 1;
25         else if (*m > T)
26             r = m - 1;
27         else
28             return m;
29     }
30
31     return A.end();
32 }
33
34 int main()
35 {
36     // declaration and assignment
37     vector<int> vec = {1, 3, 2, 8, 9, 10, 17, 15, 31};
38
39     print_vector(vec);
40     sort(vec.begin(), vec.end());
41     print_vector(vec);
42
43     vector<int>::iterator it = my_binary_search(vec, 10);
44     cout << "index : " << it - vec.begin() << ", value : " << *it << endl;
45     return 0;
46 }
```

# Outline

- STL in modern C++
  - Vector
    - Container
    - Algorithm
  - Set
  - Map

```
# include <iostream>
# include <string>
# include <vector>
# include <set>
# include <map>
# include <stack>
# include <queue>
```

# 2. Set

## 2.1. Introduction

- **Containers**

- Create and store data and objects

- Sequence containers: array, vector, list, deque, forward\_list
- Container adaptors: stack, queue, priority\_queue
- Associative container: **set**, multiset, map, multimap

### Standard Template Library in C++

#### Container

- Sequence Containers
- Associative Containers
- Container Adapters
- Unordered Associative Containers

#### Iterator

- begin()
- next()
- prev()
- advance()
- end()

#### Algorithm

- Sorting Algorithms
- Search algorithms
- Non modifying algorithms
- Modifying algorithms
- Numeric algorithms
- Minimum and Maximum operations

# 2. Set

## 2.1. Introduction

- Associative Containers
  - Elements in **associative containers** are referenced by their **key** (=value in set) and not by their absolute position in the container
- What is the Set?
  - An associative container that stores unique elements following a specific order.
  - Ordered
    - The elements in the container always follow **a strict order**.
  - Unique keys
    - No two elements in the container can have **equivalent keys**



# 2. Set

## 2.1. Introduction

- Syntax
  - `set <data_type> set_name;`
- Exercise
  - Initialize & modifier
  - Lookup



<https://www.geeksforgeeks.org/set-in-cpp-stl/>  
<https://en.cppreference.com/w/cpp/container/set.html>

### Iterators

|                                         |                                                                         |
|-----------------------------------------|-------------------------------------------------------------------------|
| <b>begin</b><br><b>cbegin</b> (C++11)   | returns an iterator to the beginning<br>(public member function)        |
| <b>end</b><br><b>cend</b> (C++11)       | returns an iterator to the end<br>(public member function)              |
| <b>rbegin</b><br><b>crbegin</b> (C++11) | returns a reverse iterator to the beginning<br>(public member function) |
| <b>rend</b><br><b>crend</b> (C++11)     | returns a reverse iterator to the end<br>(public member function)       |

### Capacity

|                 |                                                                             |
|-----------------|-----------------------------------------------------------------------------|
| <b>empty</b>    | checks whether the container is empty<br>(public member function)           |
| <b>size</b>     | returns the number of elements<br>(public member function)                  |
| <b>max_size</b> | returns the maximum possible number of elements<br>(public member function) |

### Modifiers

|                             |                                                                            |
|-----------------------------|----------------------------------------------------------------------------|
| <b>clear</b>                | clears the contents<br>(public member function)                            |
| <b>insert</b>               | inserts elements <b>or nodes</b> (since C++17)<br>(public member function) |
| <b>emplace</b> (C++11)      | constructs element in-place<br>(public member function)                    |
| <b>emplace_hint</b> (C++11) | constructs elements in-place using a hint<br>(public member function)      |
| <b>erase</b>                | erases elements<br>(public member function)                                |
| <b>swap</b>                 | swaps the contents<br>(public member function)                             |
| <b>extract</b> (C++17)      | extracts nodes from the container<br>(public member function)              |
| <b>merge</b> (C++17)        | splices nodes from another container<br>(public member function)           |

### Lookup

|                         |                                                                                                         |
|-------------------------|---------------------------------------------------------------------------------------------------------|
| <b>count</b>            | returns the number of elements matching specific key<br>(public member function)                        |
| <b>find</b>             | finds element with specific key<br>(public member function)                                             |
| <b>contains</b> (C++20) | checks if the container contains element with specific key<br>(public member function)                  |
| <b>equal_range</b>      | returns range of elements matching a specific key<br>(public member function)                           |
| <b>lower_bound</b>      | returns an iterator to the first element <i>not less</i> than the given key<br>(public member function) |
| <b>upper_bound</b>      | returns an iterator to the first element <i>greater</i> than the given key<br>(public member function)  |

# 2. Set

## 2.2. Exercise #4 – Initialization & Modifier

### Modifiers

|                             |                                                                       |                        |                                                                     |             |                                                |
|-----------------------------|-----------------------------------------------------------------------|------------------------|---------------------------------------------------------------------|-------------|------------------------------------------------|
| <b>clear</b>                | clears the contents<br>(public member function)                       | <b>insert</b>          | inserts elements or nodes (since C++17)<br>(public member function) |             |                                                |
| <b>insert_range</b> (C++23) | inserts a range of elements<br>(public member function)               | <b>emplace</b> (C++11) | constructs element in-place<br>(public member function)             |             |                                                |
| <b>emplace_hint</b> (C++11) | constructs elements in-place using a hint<br>(public member function) | <b>erase</b>           | erases elements<br>(public member function)                         | <b>swap</b> | swaps the contents<br>(public member function) |
| <b>extract</b> (C++17)      | extracts nodes from the container<br>(public member function)         | <b>merge</b> (C++17)   | splices nodes from another container<br>(public member function)    |             |                                                |

# 2. Set

## 2.2. Exercise #4 – Initialization & Modifier

- Syntax with compare class
  - `set <data_type, compare> set_name;`

```
template<
    class Key,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<Key>
> class set;
```

Key = data\_type

### Specializations

`less<void>` (C++14) function object implementing `x < y` deducing parameter and return types  
(class template specialization)

### Specializations

`greater<void>` (C++14) function object implementing `x > y` deducing parameter and return types  
(class template specialization)

- Example : `set<int, less<int>>> myset;`

# 2. Set

## 2.2. Exercise #4 – Initialization & Modifier

```
1 #include <iostream>
2 #include <iterator>
3 #include <set>
4 using namespace std;
5
6 void print_set(set<int, greater<int>> s)
7 {
8     set<int, greater<int>>::iterator it;
9     for (it = s.begin(); it != s.end(); it++)
10         cout << *it << " ";
11     cout << endl;
12 }
13
14 void print_set(set<int> s)
15 {
16     set<int>::iterator it;
17     for (it = s.begin(); it != s.end(); it++)
18         cout << *it << " ";
19     cout << endl;
20 }
21
22 int main()
23 {
24     // empty set container
25     set<int> s1;
26     if (s1.empty())
27         cout << "set s1 is empty now" << endl;
28
29     // insert elements in random order
30     s1.insert(40);
31     s1.insert(30);
32     s1.insert(50);
33     s1.insert(10);
34     s1.insert(60);
35
36     // only one 50 will be added to the set
37     s1.insert(50);
38     cout << "The set s1 is : ";
39     print_set(s1);
40 }
```

```
41 set<int, greater<int>> s2(s1.begin(), s1.end());
42 cout << "The set s2 after assign from s1 is : ";
43 print_set(s2);
44
45 cout << "The set s2 after removal of elements less than 30 is : ";
46 s2.erase(s2.begin(), s2.find(30));
47 print_set(s2);
48
49 return 0;
50 }
```

```
set s1 is empty now
The set s1 is : 10 30 40 50 60
The set s2 after assign from s1 is : 60 50 40 30 10
The set s2 after removal of elements less than 30 is : 30 10
```

예제 코드:

```
#include <iostream>
#include <functional>

int main() {
    std::greater<int> greater;
    std::cout << greater(3, 4) << std::endl; // 0
    std::cout << greater(4, 3) << std::endl; // 1

    return 0;
}
```

출력:

```
0
1
```



# 2. Set

## 2.2. Exercise #4 – Initialization & Modifier

```
1 #include <iostream>
2 #include <iterator>
3 #include <set>
4 using namespace std;
5
6 void print_set(set<int, greater<int>> s)
7 {
8     set<int, greater<int>>::iterator it;
9     for (it = s.begin(); it != s.end(); it++)
10         cout << *it << " ";
11     cout << endl;
12 }
13
14 void print_set(set<int> s)
15 {
16     set<int>::iterator it;
17     for (it = s.begin(); it != s.end(); it++)
18         cout << *it << " ";
19     cout << endl;
20 }
21
22 int main()
23 {
24     // empty set container
25     set<int> s1;
26     if (s1.empty())
27         cout << "set s1 is empty now" << endl;
28
29     // insert elements in random order
30     s1.insert(40);
31     s1.insert(30);
32     s1.insert(50);
33     s1.insert(10);
34     s1.insert(60);
35
36     // only one 50 will be added to the set
37     s1.insert(50);
38     cout << "The set s1 is : ";
39     print_set(s1);
40 }
```

```
41 set<int, greater<int>> s2(s1.begin(), s1.end());
42 cout << "The set s2 after assign from s1 is : ";
43 print_set(s2);
44
45 cout << "The set s2 after removal of elements less than 30 is : ";
46 s2.erase(s2.begin(), s2.find(30));
47 print_set(s2);
48
49 return 0;
50 }
```

```
set s1 is empty now
The set s1 is : 10 30 40 50 60
The set s2 after assign from s1 is : 60 50 40 30 10
The set s2 after removal of elements less than 30 is : 30 10
```

예제 코드:

```
#include <iostream>
#include <functional>

int main() {
    std::greater<int> greater;
    std::cout << greater(3, 4) << std::endl; // 0
    std::cout << greater(4, 3) << std::endl; // 1

    return 0;
}
```

출력:

```
0
1
```



# 2. Set

## 2.2. Exercise #5 – Lookup

### Lookup

|                    |                                                                                                         |             |                                                             |
|--------------------|---------------------------------------------------------------------------------------------------------|-------------|-------------------------------------------------------------|
| <b>count</b>       | returns the number of elements matching specific key<br>(public member function)                        | <b>find</b> | finds element with specific key<br>(public member function) |
| <b>contains</b>    | (C++20)<br>checks if the container contains element with specific key<br>(public member function)       |             |                                                             |
| <b>equal_range</b> | returns range of elements matching a specific key<br>(public member function)                           |             |                                                             |
| <b>lower_bound</b> | returns an iterator to the first element <i>not less</i> than the given key<br>(public member function) |             |                                                             |
| <b>upper_bound</b> | returns an iterator to the first element <i>greater</i> than the given key<br>(public member function)  |             |                                                             |

# 2. Set

## 2.2. Upper and Lower Bound

- [lower\\_bound\(const g\)](#) – Returns an iterator to the first element that is **equivalent to 'g' or definitely will not go before the element 'g'** in the set.
- [upper\\_bound\(const g\)](#) – Returns an iterator to the first element that **will go after the element 'g'** in the set.

```
s1.erase(50): 1 removed
The set s1 after removal of 50 is : 10 20 30 40 60

s1.lower_bound(40) : 40
s1.upper_bound(40) : 60
size of s1 is: 5
find 60 in s1 is : 60
find 70 in s1 is : 20
count 60 in s1 is : 1
count 70 in s1 is : 0
```

# 2. Set

## 2.2. Exercise #5 – Lookup

```
15 int main()
16 {
17     set<int> s1 = {10, 20, 30, 40, 50, 60};
18
19     int num;
20     ?
21     cout << "s1.erase(50): " << num << " removed " << endl
22         << "The set s1 after removal of 50 is : ";
23     print_set(s1);
24
25     cout << endl
26         << "s1.lower_bound(40) : " << ? << endl
27         << "s1.upper_bound(40) : " << ? << endl;
28
29     cout
30         << "size of s1 is: " << s1.size() << endl
31         << "find 60 in s1 is : " << ? << endl
32         << "find 70 in s1 is : " << ? << endl
33         << "count 60 in s1 is : " << ? << endl
34         << "count 70 in s1 is : " << ? << endl;
35
36     return 0;
```

```
s1.erase(50): 1 removed
The set s1 after removal of 50 is : 10 20 30 40 60

s1.lower_bound(40) : 40
s1.upper_bound(40) : 60
size of s1 is: 5
find 60 in s1 is : 60
find 70 in s1 is : 5
count 60 in s1 is : 1
count 70 in s1 is : 0
```

Find 70 : No 70 exists -> return s1.end()  
5 : dummy value



# 2. Set

## 2.2. Exercise #5 – Lookup

```
15 int main()
16 {
17     set<int> s1 = {10, 20, 30, 40, 50, 60};
18
19     int num;
20     num = s1.erase(50);
21     cout << "s1.erase(50): " << num << " removed " << endl
22         << "The set s1 after removal of 50 is : ";
23     print_set(s1);
24
25     cout << endl
26         << "s1.lower_bound(40) : " << *s1.lower_bound(40) << endl
27         << "s1.upper_bound(40) : " << *s1.upper_bound(40) << endl;
28
29     cout
30         << "size of s1 is: " << s1.size() << endl
31         << "find 60 in s1 is : " << *s1.find(60) << endl
32         << "find 70 in s1 is : " << *s1.find(70) << endl
33         << "count 60 in s1 is : " << s1.count(60) << endl
34         << "count 70 in s1 is : " << s1.count(70) << endl;
35
36     return 0;
```

```
s1.erase(50): 1 removed
The set s1 after removal of 50 is : 10 20 30 40 60

s1.lower_bound(40) : 40
s1.upper_bound(40) : 60
size of s1 is: 5
find 60 in s1 is : 60
find 70 in s1 is : 5
count 60 in s1 is : 1
count 70 in s1 is : 0
```

Find 70 : No 70 exists -> return s1.end()  
5 : dummy value

```
if (s1.find(70) == s1.end()){
    cout << "s1.find(70) == s1.end() - pointing same position\n";
}
```

s1.find(70) == s1.end() - pointing same position

# Outline

- STL in modern C++
  - Vector
    - Container
    - Algorithm
  - Set
  - Map

# 3. Map

## 3.1. Introduction

- **Containers**

- Create and store data and objects

- Sequence containers: array, vector, list, deque, forward\_list
- Container adaptors: stack, queue, priority\_queue
- Associative container: set, multiset, **map**, multimap

### Standard Template Library in C++

#### Container

- Sequence Containers
- Associative Containers
- Container Adapters
- Unordered Associative Containers

#### Iterator

- begin()
- next()
- prev()
- advance()
- end()

#### Algorithm

- Sorting Algorithms
- Search algorithms
- Non modifying algorithms
- Modifying algorithms
- Numeric algorithms
- Minimum and Maximum operations

# 3. Map

## 3.1. Introduction

- What is the Map?
  - The container that store elements formed by a **combination of a key-value pair**, following a specific order
  - Properties
    - Associative & Ordered & Unique keys (==set)
  - Map
    - Each element associates a key to a mapped value
    - Keys are meant to identify the elements whose main content is the mapped value.
  - Unique keys
    - No two elements in the container can have equivalent keys

# 3. Map

## 3.1. Intro

- Syntax
  - `map<data_type of key, data_type of value> map_name;`

### Capacity

|                 |                                                                             |
|-----------------|-----------------------------------------------------------------------------|
| <b>empty</b>    | checks whether the container is empty<br>(public member function)           |
| <b>size</b>     | returns the number of elements<br>(public member function)                  |
| <b>max_size</b> | returns the maximum possible number of elements<br>(public member function) |

### Lookup

|                         |                                                                                                         |
|-------------------------|---------------------------------------------------------------------------------------------------------|
| <b>count</b>            | returns the number of elements matching specific key<br>(public member function)                        |
| <b>find</b>             | finds element with specific key<br>(public member function)                                             |
| <b>contains</b> (C++20) | checks if the container contains element with specific key<br>(public member function)                  |
| <b>equal_range</b>      | returns range of elements matching a specific key<br>(public member function)                           |
| <b>lower_bound</b>      | returns an iterator to the first element <i>not less</i> than the given key<br>(public member function) |
| <b>upper_bound</b>      | returns an iterator to the first element <i>greater</i> than the given key<br>(public member function)  |

### Modifiers

|                             |                                                                       |
|-----------------------------|-----------------------------------------------------------------------|
| <b>clear</b>                | clears the contents<br>(public member function)                       |
| <b>insert</b>               | inserts elements or nodes (since C++17)<br>(public member function)   |
| <b>emplace</b> (C++11)      | constructs element in-place<br>(public member function)               |
| <b>emplace_hint</b> (C++11) | constructs elements in-place using a hint<br>(public member function) |
| <b>erase</b>                | erases elements<br>(public member function)                           |
| <b>swap</b>                 | swaps the contents<br>(public member function)                        |
| <b>extract</b> (C++17)      | extracts nodes from the container<br>(public member function)         |
| <b>merge</b> (C++17)        | splices nodes from another container<br>(public member function)      |



# 3. Map

## 3.2. Exercise #6 – Map

```
1 #include <iostream>
2 #include <iterator>
3 #include <map>
4 using namespace std;
5 void print_map(map<int, int> &m)
6 {
7     cout << "\\tkey: \\t";
8     for (auto i : m)
9         cout << i.first << "\\t";
10    cout << endl;
11    cout << "\\tvalue: \\t";
12    for (auto i : m)
13        cout << i.second << "\\t";
14    cout << endl;
15 }
16 int main()
17 {
18     map<int, int> m1;
19     // insert elements
20     m1.insert(pair<int, int>(2, 30));
21     m1.insert(pair<int, int>(1, 40));
22     m1.insert(pair<int, int>(3, 60));
23     m1.insert(pair<int, int>(4, 20));
24     m1.insert(pair<int, int>(1, 30)); // already exist
25     cout << "The map m1 is: " << endl;
26     print_map(m1);
27     // upper and lower bound
28     cout << "m1.lower_bound(3) : "
29         << "\\tkey: " << ? << "\\tvalue: " << m1
30     cout << "m1.upper_bound(3) : "
31         << "\\tkey: " << ? << "\\tvalue: " << m1
32     cout << "count : " << ? << endl;
33     cout << "size : " << ? << endl;
34
35     cout << "The map m2 after assign from m1 is: " << endl;
36     ? m2(m1.begin(), m1.end());
37     print_map(m2);
38
39     cout << "m2 after removal of elements less than key=2 : " << endl;
40     m2.erase(m2.begin(), m2.find(2));
41     print_map(m2);
42     return 0;
43 }
```

The map m1 is:

| key:   | 1  | 2  | 3  | 4  |
|--------|----|----|----|----|
| value: | 40 | 30 | 60 | 20 |

m1.lower\_bound(3) : ?

m1.upper\_bound(3) : ?

count : 1

size : 4

The map m2 after assign from m1 is:

| key:   | ? |
|--------|---|
| value: | ? |

m2 after removal of elements less than key=2 :

| key:   | ? |
|--------|---|
| value: | ? |

# 3. Map

## 3.2 Exercise #6 – Map

```
1 #include <iostream>
2 #include <iterator>
3 #include <map>
4 using namespace std;
5 void print_map(map<int, int> &m)
6 {
7     cout << "\tkey: \t";
8     for (auto i : m)
9         cout << i.first << "\t";
10    cout << endl;
11    << "\tvalue: \t";
12    for (auto i : m)
13        cout << i.second << "\t";
14    cout << endl;
15 }
16 int main()
17 {
18     map<int, int> m1;
19     // insert elements
20     m1.insert(pair<int, int>(2, 30));
21     m1.insert(pair<int, int>(1, 40));
22     m1.insert(pair<int, int>(3, 60));
23     m1.insert(pair<int, int>(4, 20));
24     m1.insert(pair<int, int>(1, 30)); // already exist
25     cout << "The map m1 is: " << endl;
26     print_map(m1);
27     // upper and lower bound
28     cout << "m1.lower_bound(3) : "
29         << "\tkey: " << m1.lower_bound(3)->first << "\tvalue: " << m1.lower_bound(3)->second << endl;
30     cout << "m1.upper_bound(3) : "
31         << "\tkey: " << m1.upper_bound(3)->first << "\tvalue: " << m1.upper_bound(3)->second << endl;
32     cout << "count : " << m1.count(1) << endl;
33     cout << "size : " << m1.size() << endl;
34 }
```

```
The map m1 is:
key:      1      2      3      4
value:    40     30     60     20
m1.lower_bound(3) : key: 3 value: 60
m1.upper_bound(3) : key: 4 value: 20
count : 1
size : 4
```

```
The map m2 after assign from m1 is:
key:      1      2      3      4
value:    40     30     60     20
m2 after removal of elements less than key=2 :
key:      2      3      4
value:    30     60     20
```

```
cout << "The map m2 after assign from m1 is: " << endl;
map<int, int> m2(m1.begin(), m1.end());
print_map(m2);

cout << "m2 after removal of elements less than key=2 : " << endl;
m2.erase(m2.begin(), m2.find(2));
print_map(m2);
return 0;
```

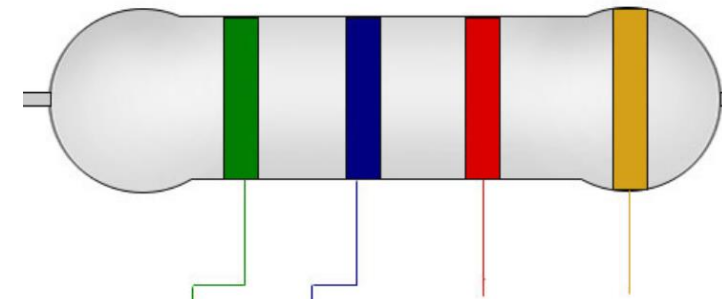
# 4.Attendance check

- Submit your attendance check by October 13 (Today), 18:00



# 4. Assignment Resistance

- The color band in resistor represents its resistance,
  - $R = (10 * \text{first} + \text{second}) * \text{multiplier}(\Omega)$
  - Color-digit mapping is given in the figure
- Example)
  - Green / Blue / Red:
    - $R = (10 * 5 + 6) * 100 = 5600\Omega$
  - Yellow / Violet / Blue:
    - $R = (10 * 4 + 7) * 1M = 47M\Omega$



|        | First Digit | Second Digit | Multiplier |
|--------|-------------|--------------|------------|
| Black  | Nil         | 0            | 1          |
| Brown  | 1           | 1            | 10         |
| Red    | 2           | 2            | 100        |
| Orange | 3           | 3            | 1000       |
| Yellow | 4           | 4            | 10000      |
| Green  | 5           | 5            | 100000     |
| Blue   | 6           | 6            | 1M         |
| Violet | 7           | 7            | 10M        |
| Grey   | 8           | 8            | 100M       |
| White  | 9           | 9            | 1G         |

# 4. Assignment Resistance

- Write a program that
  - Convert resistance  $\Omega$  to three colors

```
Type your mode (0: R->C, 1: C->R): 0
Type the resistance: 4200
Three colors for that resistance is: Yellow, Red, Red
```

- Convert three colors to resistance  $\Omega$

```
Type your mode (0: R->C, 1: C->R): 1
Type each color of three bands: Yellow Red Red
Resistance for that three colors is: 4200
```

- We offer a skeleton code. All you need to do is to complete only two functions.

# 4. Assignment Resistance

```
1 #include <iostream>
2 #include <map>
3 #include <string>
4 #include <cmath>
5 #include <tuple>
6 using namespace std;
7 int color_to_resistance(tuple<string, string, string> &tup, map<string, int> &colormap)
8 {
9     int R;
10    // Implement here
11
12    return R;
13 }
14
15 tuple<string, string, string> resistance_to_color(int R, map<string, int> &colormap)
16 {
17     string s1, s2, s3;
18    // Implement here
19
20    return make_tuple(s1, s2, s3);
21 }
22
```

```
23 // Do Not Fix main function: Only implement above functions.
24 int main()
25 {
26     int n;
27     string str1, str2, str3;
28     map<string, int> colormap;
29     colormap = {
30         {"Black", 0},
31         {"Brown", 1},
32         {"Red", 2},
33         {"Orange", 3},
34         {"Yellow", 4},
35         {"Green", 5},
36         {"Blue", 6},
37         {"Violet", 7},
38         {"Grey", 8},
39         {"White", 9},
40     };
41     cout << "Type your mode (0:R->C, 1:C->R): ";
42     cin >> n;
43     if (n == 0)
44     {
45         cout << "Type the resistance: ";
46         cin >> n;
47         auto colors = resistance_to_color(n, colormap);
48         cout << "Three colors for that resistnace is: " << get<0>(colors) << ", " << get<1>(colors) << ", " << get<2>(colors) << endl;
49     }
50     if (n == 1)
51     {
52         cout << "Type each color of three bands: ";
53         cin >> str1 >> str2 >> str3;
54         auto colors = make_tuple(str1, str2, str3);
55         cout << "Resistances for that three colors is: " << color_to_resistance(colors, colormap) << endl;
56     }
57     return 0;
58 }
```

submit due date: 10/20 15:00

```
Type your mode (0: R->C, 1: C->R): 0
Type the resistance: 4200
Three colors for that resistance is: Yellow, Red, Red
```

```
Type your mode (0: R->C, 1: C->R): 1
Type each color of three bands: Yellow Red Red
Resistance for that three colors is: 4200
```